AdAgentFlow-RT: A Contractual Runtime for Reliable High-Throughput Long-Horizon Multi-Agent Workflows

Anonymous Authors
Anonymous Institution
anonymous@example.com

ABSTRACT

Existing agent benchmarks primarily measure whether an agent completes a task. This paper studies a complementary question: whether a long-horizon multi-agent runtime remains reliable, bounded, diagnosable, and recoverable under concurrent production-style execution. We present AdAgentFlow-RT, a contractual runtime that represents workflows as contract-bearing execution graphs, monitors schema, semantic, dependency, resource, and recovery contracts, localizes faults, applies bounded recovery, and records event-sourced traces. We do not introduce a new task dataset. Instead, we build a production-stress evaluation harness on top of public workloads from tau2-bench / tau3-bench and AgentChangeBench. The harness preserves benchmark-native metrics while adding runtime stability metrics such as dead-letter rate, recovery success, retry amplification, silent failure rate, fault propagation depth, contaminated artifact count, and mean time to detect and recover. A runnable smoke harness demonstrates the end-to-end instrumentation path with deterministic tasks, fault injection, ablations, JSONL traces, aggregated CSV summaries, and paper-facing figures.

CCS CONCEPTS

• **Computing methodologies** → **Multi-agent systems**; • **Software and its engineering** → *Runtime environments.*

KEYWORDS

multi-agent systems, runtime reliability, fault injection, runtime verification, agent benchmarks

ACM Reference Format:
Anonymous Authors. 2026. AdAgentFlow-RT: A Contractual Runtime for Reliable High-Throughput Long-Horizon Multi-Agent Workflows. In *Proceedings of ACM Conference (Conference'17).* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Long-horizon agent systems increasingly combine planning, tool use, memory, verification, and multiple specialized agents. Public benchmarks have improved the field's ability to measure task completion, but production deployments face a broader reliability problem: the runtime must bound cost, detect contract violations, avoid silent failures, contain faulty artifacts, recover without unbounded retry loops, and preserve traces that support diagnosis.

AdAgentFlow-RT reframes an existing multi-agent engineering prototype as a contractual runtime for reliable high-throughput multi-agent workflows. The prior advertising generation workflow remains an example domain plugin. The paper contribution is a reusable runtime abstraction and a stress evaluation harness that can wrap public task environments without changing their task data.

This work makes three claims. First, runtime contracts should cover schemas, semantics, dependencies, budgets, and recovery policies, not only JSON validity. Second, production reliability should be evaluated under controlled concurrency and fault injection on top of public task benchmarks. Third, event-sourced traces and bounded recovery make failures more diagnosable and reduce retry amplification compared with vanilla, retry-only, and schema-only baselines.

The key distinction from a benchmark paper is that the workload is not new. tau2-bench / tau3-bench and AgentChangeBench provide public task environments and native evaluation signals [1, 3]. AdAgentFlow-RT contributes the execution substrate around those tasks: contract enforcement, fault localization, bounded recovery, dead-letter handling, and trace-derived reliability metrics.

This paper makes the following contributions:

- A contractual execution model for long-horizon multi-agent workflows, including contracts over artifacts, dependencies, resources, and recovery behavior.
- A production-oriented runtime kernel with a scheduler, monitor, artifact store, fault localizer, recovery controller, and event-sourced trace.
- A stress evaluation harness that runs comparable runtime strategies under the same tasks, models, tools, fault distributions, concurrency, and recovery budgets.
- A reliability metric suite that complements benchmark-native success with dead-letter rate, recovery success, retry amplification, silent failure rate, fault propagation depth, contaminated artifact count, and mean time to detect and recover.

The current artifact includes a deterministic smoke harness and external command planning for tau2-bench / tau3-bench and AgentChangeBench. Full benchmark-scale experiments are planned after smoke validation, following the protocol in Section 5.

2 BACKGROUND AND MOTIVATION

Multi-agent workflows fail in ways that are not captured by final task success alone. A workflow can complete while using stale context, propagating a contaminated artifact, violating a resource budget, silently accepting a malformed tool response, or retrying until cost becomes unacceptable.

Conference'17, July 2017, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Retry and prompt repair are useful but insufficient as the only reliability mechanism. They do not identify the responsible node, downstream contamination, recovery budget, or whether the runtime should rollback, reroute, degrade, escalate, or dead-letter a task.

2.1 Failure Modes

We focus on failures that appear at runtime boundaries:

- **Contract failures**: malformed outputs, schema drift, missing artifacts, failed preconditions, or semantic mismatch.
- **Coordination failures**: duplicate node execution, stale context, missing dependency artifacts, and inconsistent downstream state.
- **Resource failures**: latency, token, tool-call, or retry budgets exceeded during long-horizon execution.
- **Environment failures**: tool timeouts, rate limits, partial responses, and worker crashes.
- **Silent failures**: runtime success states that disagree with benchmark-native evaluation.

These failures motivate runtime-level guarantees that are orthogonal to model capability. A stronger model may complete more tasks, but a production runtime still needs traceability, bounded recovery, and controlled failure containment.

2.2 Why Public Benchmarks Are Still Needed

The harness deliberately uses public benchmarks instead of creating a new dataset. Public tasks provide comparability and established native metrics. The harness adds stress dimensions around them: concurrency, repeated trials, injected tool faults, schema drift, duplicate execution, stale context, and recovery budgets. This separation lets the paper ask a runtime question without weakening the benchmark question.

3 CONTRACTUAL EXECUTION MODEL

AdAgentFlow-RT models a multi-agent workflow as a Contractual Execution Graph. Nodes declare a role, capability, agent provider, and contract name. Edges declare artifact dependencies. Contracts define input and output schemas, preconditions, postconditions, semantic checks, resource budgets, recovery policies, and escalation policies.

The first implementation supports DAG execution, including sequential paths, fan-out, joins, fallback branches, and rollback targets. Each produced artifact carries provenance, validation status, and downstream consumers. This makes failure propagation explicit and measurable.

3.1 Contracts

A contract binds a runtime capability to observable obligations:

- **Schema contract**: validates inputs and outputs.
- **Semantic contract**: captures domain invariants such as policy compliance or goal alignment.
- **Dependency contract**: requires upstream artifacts to exist and be valid before a node runs.
- **Budget contract**: bounds latency, LLM calls, tool calls, tokens, and retries.
- **Recovery contract**: lists permitted recovery actions and escalation behavior.

The current implementation represents contracts as typed Python dataclasses and supports JSON-Schema validation with a dependency-light fallback validator for minimal environments.

3.2 Artifacts and Propagation

Every node produces zero or more artifacts. An artifact records its producer, contract name, content, validation status, provenance, and downstream consumers. When a contract violation occurs, the runtime localizes the responsible node and traverses the graph to estimate affected downstream nodes and contaminated artifacts. These quantities directly support the metrics *fault propagation depth* and *contaminated artifact count*.

3.3 Bounded Recovery

Recovery is selected from a bounded policy rather than an open-ended retry loop. The controller currently supports quick repair, retry, mock reroute, rollback target selection, downstream invalidation, degradation, human escalation, and dead-letter. A recovery decision records the selected action, target node, reason, budget cost, and whether execution may continue.

4 ADAGENTFLOW-RT RUNTIME

The runtime contains a contract registry, scheduler, monitor, artifact store, fault localizer, recovery controller, event log, and replay utilities. Before a node runs, the monitor checks dependency and precondition contracts. After execution, it checks schema and budget contracts and emits violations. The fault localizer maps violations into operational classes such as artifact, coordination, resource, tool-environment, runtime, and verifier faults. The recovery controller selects bounded actions such as quick repair, retry, reroute, rollback, invalidation, degradation, human escalation, or dead-letter.

Every event is traceable by task identifier and run identifier. Events include graph creation, contract checks, node scheduling, artifact production, violations, fault diagnoses, recovery decisions, checkpoints, rollbacks, dead letters, and task finalization.

4.1 Runtime Kernel

Figure 1 shows the runtime components. The contract registry stores executable contract specifications. The graph planner constructs a dependency graph. The scheduler selects runnable nodes whose dependencies are satisfied. The monitor checks contracts before and after node execution. The artifact store records produced artifacts and validation status. The fault localizer maps violations into operational fault classes. The recovery controller selects bounded actions from policy and remaining budget. The event log records each runtime transition for replay and metric extraction.

4.2 Event-Sourced Trace

The event log records 'graph.created', 'contract.loaded', 'node.started', 'contract.checked', 'fault.injected', 'contract.violated', 'fault.localized', 'recovery.started', 'recovery.selected', 'recovery.succeeded', 'artifact.produced', 'dead_letter.created', and 'task.finalized'. Each event carries 'task_id',

Figure 1: AdAgentFlow-RT contractual runtime architecture
component=Contract Registry, role=load contracts
component=Execution Graph, role=schedule dependencies
component=Runtime Monitor, role=detect violations
component=Fault Localizer, role=classify and contain faults
component=Recovery Controller, role=select bounded recovery
component=Event Log, role=trace and replay
component=Stress Harness, role=evaluate reliability

Figure 1: AdAgentFlow-RT contractual runtime components.

Figure 2: Production-stress evaluation harness
stage=1, name=benchmark task
stage=2, name=runtime adapter
stage=3, name=stress layer
stage=4, name=runtime strategy
stage=5, name=benchmark environment
stage=6, name=benchmark + runtime evaluator

Figure 2: Production-stress harness layered around public benchmark tasks.

'run_id', optional 'node_id', status, payload, and timestamp. The smoke harness also records relative millisecond offsets so trace metrics can estimate mean time to detect and recover.

4.3 Runtime Strategies

The harness implements four comparable strategies:

- **Vanilla**: directly executes the benchmark-style task without runtime contracts or recovery.
- **Retry-only**: retries after failure until a bounded retry count is exhausted.
- **Schema-only**: validates schema and applies repair/retry, but does not localize graph-level faults.
- **AdAgentFlow-RT**: applies contract monitoring, fault localization, bounded recovery, artifact invalidation hooks, dead-letter handling, and event trace.

5 PRODUCTION-STRESS EVALUATION HARNESS

The harness wraps public benchmark tasks without modifying their data. tau2-bench / tau3-bench provide the main workload across airline, retail, and telecom domains. AgentChangeBench provides supplementary goal-change recovery evaluation across airline and retail domains.

We compare vanilla tool calling, retry-only execution, schema-only validation, and AdAgentFlow-RT. Stressors include tool timeouts, rate limits, partial responses, schema drift, stale context, duplicate messages, and worker crashes. All experiments write JSONL records with benchmark-native metrics, runtime metrics, injected faults, events, and recovery outcomes.

Figure 2 summarizes the evaluation path. A benchmark task is passed to a runtime adapter, stressors perturb tool or context boundaries, a runtime strategy executes the task, and benchmark-native and runtime-native evaluators produce a unified JSONL record.

5.1 Benchmark Adapters

The tau2-bench / tau3-bench adapter supports 'benchmark_repo_path', domain, task identifiers, number of tasks, trials, agent model, user model, and concurrency. If the external benchmark checkout is absent, the harness fails with a clear diagnostic in external mode or uses deterministic mock tasks in smoke mode. The AgentChangeBench adapter preserves the native metric slots TSR, TUE, TCRR, and GSRT.

5.2 Stressors

Stressors share a reproducible interface with a rate and seed. The implemented stressors simulate timeouts, rate limits, partial responses,

Table 1: Smoke runtime stability metrics are exported to 'runtime_stability_metrics.csv'.

Method	Viol.	Silent	Prop.	Recover ms
AdAgentFlow-RT	1.0	0.0	2	300
w/o monitor	0.0	1.0	0	0
Retry-only	0.0	0.0	0	0

schema drift, stale context, duplicate messages, and worker crashes. Each injected fault is recorded in the JSONL row and, for AdAgentFlow-RT, in the runtime trace.

5.3 Metrics

The harness reports benchmark-native task success and policy metrics, plus runtime-stability metrics:

- throughput, p50/p95/p99 latency, dead-letter rate, and cost per successful task;
- recovery success rate, contract violation rate, retry amplification, extra tool calls, and extra LLM calls;
- silent failure rate, fault propagation depth, contaminated artifact count, mean time to detect, and mean time to recover.

Every experiment writes machine-readable JSONL. Aggregation scripts produce JSON and CSV summaries, figure data, and paper-facing tables.

6 EXPERIMENTS

The smoke setting runs three tasks per domain, one trial, concurrency one, and all four runtime methods. We use this setting to validate adapter execution, result persistence, trace generation, aggregation, plotting, and ablation scripts before any full-scale benchmark run.

The main setting varies concurrency, fault rate, domains, trials, and runtime strategy. The intended matrix uses tau2-bench / tau3-bench over airline, retail, and telecom with 20–30 tasks per domain, three trials, concurrency 1/5/10/20, and fault rates 0/0.1/0.2. A compressed matrix uses airline and retail, 20 tasks per domain, two trials, concurrency 1/10/20, and fault rates 0/0.2.

6.1 Smoke Results

The current repository includes smoke artifacts produced by the deterministic harness. The results are not full benchmark claims; they validate that runtime traces, stressors, metrics, figures, and tables are generated end-to-end.

The schema-drift ablation illustrates the intended diagnostic distinction. Full AdAgentFlow-RT detects schema drift, localizes it as an artifact fault, applies quick repair, and records nonzero detection and recovery times. The 'without contract monitor' ablation marks the runtime task as successful while benchmark-native task success is false, producing a silent failure rate of 1.0 in the smoke artifact.

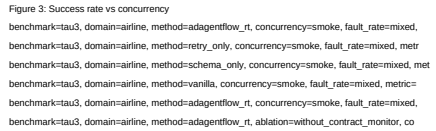

Figure 3: Success-rate figure artifact generated from aggregated summaries.

6.2 Ablation Plan

The minimum ablation set includes full AdAgentFlow-RT, without contract monitor, without fault localizer, without bounded recovery, without event trace, and retry-only. Planned extended ablations remove semantic contracts, rollback/invalidation, and backpressure once the external benchmark matrix is running.

6.3 External Benchmark Runs

External command planning is implemented for tau2-bench / tau3-bench and AgentChangeBench. When a benchmark checkout is provided, the adapter records the planned command in the JSONL output and separates those rows from deterministic mock rows using 'benchmark_adapter_mode'. The next experimental step is to execute these planned commands against installed benchmark checkouts and replace smoke-only figure data with full benchmark results.

7 DISCUSSION

The contractual runtime trades additional monitoring and trace overhead for improved diagnosability and bounded recovery. The key limitation is that semantic checks require domain-specific verifiers or benchmark-specific policies. The harness is designed so stronger verifiers can be added without changing workload data.

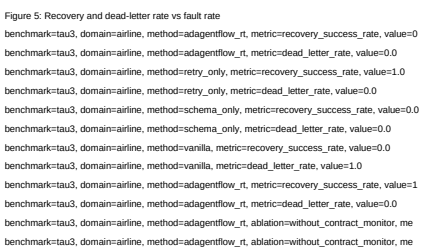

Figure 4: Recovery and dead-letter artifact generated from fault-injection summaries.

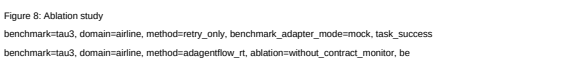

Figure 5: Ablation figure artifact generated from smoke summaries.

7.1 Cost and Reliability

Contracts and traces add runtime work. The relevant tradeoff is not raw latency alone, but cost per successful task under fault injection. A retry-only strategy may appear simple, but can amplify tool and LLM calls without identifying the contaminated artifact. AdAgentFlow-RT pays monitoring overhead to reduce silent failures and make recovery decisions inspectable.

7.2 Limits of the Current Artifact

The current repository contains a deterministic smoke harness and external benchmark command planning. It does not yet claim full tau2-bench / tau3-bench or AgentChangeBench results. Full claims require installed benchmark checkouts, configured models, repeated trials, and the main experiment matrix. The smoke artifacts are included to verify that the harness writes JSONL rows, aggregates metrics, exports figures, and produces paper tables.

7.3 Threats to Validity

Fault distributions may not match all production incidents. Semantic contracts can be incomplete or domain-biased. Some recovery actions, such as rerouting and rollback, are currently implemented as in-memory skeletons rather than deeply integrated with the existing database-backed orchestrator. These limits are acceptable for the current runtime-kernel phase, but must be addressed before making production claims.

8 RELATED WORK

This work relates to agent benchmarks, multi-agent frameworks, runtime verification, workflow orchestration, fault injection, chaos engineering, and production observability. Unlike task-only benchmark evaluations, AdAgentFlow-RT focuses on operational reliability under stress.

8.1 Agent Benchmarks

Agent benchmarks evaluate task completion, tool-use correctness, policy compliance, and goal-change behavior. This work treats tau2-bench / tau3-bench and AgentChangeBench [1, 3] as workload providers rather than datasets to replace. The harness preserves native metrics and adds runtime-stability metrics around the same tasks.

8.2 Multi-Agent Frameworks

Multi-agent frameworks provide abstractions for agent composition, tool invocation, message routing, planning, and memory [4,

5]. AdAgentFlow-RT complements these capabilities with execution contracts, bounded recovery, event traces, dead-letter behavior, and stress evaluation.

8.3 Runtime Verification and Observability

Runtime verification checks whether executions satisfy specified properties. AdAgentFlow-RT adapts this view to probabilistic agent workflows by making contracts executable and recovery-aware. Production observability motivates the event-sourced trace: every violation, diagnosis, and recovery decision must be attributable to a task, run, and node.

8.4 Fault Injection

Fault injection and chaos engineering test whether a system remains reliable under controlled failures [2]. The harness applies those ideas to multi-agent runtime boundaries: tool calls, context, schema, worker execution, and duplicate delivery.

9 CONCLUSION

AdAgentFlow-RT evaluates and improves the runtime reliability of long-horizon multi-agent workflows. Rather than introducing a new dataset, it layers production stress, contractual monitoring, fault localization, bounded recovery, and event-sourced tracing on top of public benchmarks. The current artifact establishes the runtime kernel, stress harness, baselines, stressors, trace-derived metrics, ablations, figures, and paper tables. The next step is to execute the external benchmark matrix and replace smoke artifacts with full tau2-bench / tau3-bench and AgentChangeBench results.

REFERENCES

[1] AgentChangeBench Contributors. 2026. AgentChangeBench. Public benchmark used as a supplementary goal-change workload.
[2] Casey Rosenthal, Nora Jones, Benjamin Daly, and Aaron Blohowiak. 2020. *Chaos Engineering: System Resiliency in Practice.* O'Reilly Media.
[3] Sierra Research. 2026. tau2-bench: Public Task Environments for Agent Evaluation. https://github.com/sierra-research/tau2-bench.
[4] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed H. Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155 [cs.AI]
[5] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.